

Descrizione Progetto ASD - 13/09

Giovanni Forlani

September 2026

1 Introduzione

L'obiettivo del progetto è quello di creare un codice che permette di trovare il costo di un cammino minimax ottimo in un grafo non orientato. Per sviluppare questo codice si usano principalmente grafi e tabelle hash come strutture di dati.

L'idea è quella di creare un grafo composto da Autonomous Systems (AS) come vertici. Gli archi fra i vertici descrivono la relazione: "un pacchetto di dati è passato tra questi due AS". Il peso per ciascun arco è dato dal numero di pacchetti di dati che transitano tra due AS: il peso $w(u, v)$ dell'arco non orientato u, v è uguale al numero di volte che compare l'arco u, v nel dataset considerato, che chiamerò $f(u, v)$.

Si considera quindi un grafo non orientato connesso (se non è connesso si considera la componente connessa più grande). Dati due nodi u, v , si cerca il minimo della funzione costo al variare dei cammini $P = (u_0 = u, u_1, \dots, u_k = v)$ che congiungono u e v , dove la funzione costo è data da

$$\text{costo}(P) = \max_{0 \leq i < k} f(u_i, u_{i+1}) \quad (1)$$

2

2.1 Costruzione del grafo

Il grafo sarà composto da:

- Nodi: contengono il numero identificativo dell'AS e gli archi adiacenti
- Archi: contengono il primo nodo u , il secondo nodo v , e il peso $f(u, v)$

2.2 Gestione dell'input

- Leggo un file `.all-paths.bz2`;
- creo un parser che restituisca la parte delle stringhe "che mi interessa" in un vettore;
- leggo la prima entrata del vettore, se è un nuovo numero identificativo, creo il nodo, altrimenti passo all'entrata successiva;
- leggo la seconda entrata del vettore, se è un numero nuovo creo il nodo, altrimenti mi concentro direttamente sull'arco; se non esiste l'arco che connette i due nodi lo creo (con $f(u, v) = 1$), altrimenti aggiorno $f(u, v)$

$$f(u, v) = f(u, v) + 1 \quad (2)$$

- continuo il procedimento fino a che non ho letto tutto il vettore che rappresenta il cammino.

2.3 Funzioni da implementare

2.3.1 Grafo

- `AggiungiNodo`: legge il numero identificativo, controlla se esiste già nel grafo e se non esiste crea il nuovo nodo;
- `AggiungiArco`: crea il collegamento tra due nodi se non esiste, se esiste aggiorna $f(u, v)$;
- `Cammino`: estrapola le coppie consecutive di AS da una stringa
- `CaricaInput`: legge il file, contiene il parser che individua il cammino e chiama `Cammino`;
- `Vicini`: per l'accesso ai vicini di un nodo;

2.3.2 Connessione

Per questa parte del codice, vorrei verificare la connessione del grafo creato e salvarmi la componente connessa maggiore. Ho quindi bisogno di

- `DFS`;
- `ComponenteMassima`: salva la componente che sta leggendo e la confronta con quella salvata come componente più grande attuale, restituisce la componente massima.

2.3.3 Attribuzione pesi

Pensavo di scrivere una funzione separata che salva i pesi degli archi, ma scrivendo ho capito di poter inserire questa parte direttamente nella costruzione del grafo.

2.3.4 Funzioni per cammino minimax

Implemento la funzione `EsisteCammino`, per verificare se esiste un cammino di peso non superiore alla soglia individuata che collega due AS, identificati con il loro indice interno. Poi inserisco la funzione `PesiDistinti`, che crea il vettore con tutti i pesi presenti nel grafo, e lo ordina con un merge sort.

2.4 Cammino minimax

La prima idea per trovare il minimo del costo dei cammini che congiungono due nodi u e v consiste nel fissare una soglia, tenere il grafo con archi di peso minore o uguale alla soglia e chiedersi se u e v sono ancora collegati da un cammino. In caso affermativo posso migliorare la soglia e ripetere il procedimento, altrimenti allento un po' la soglia e ripeto comunque il procedimento.

Nello specifico, potrei inserire tutti i pesi nel grafo in un array in ordine crescente, scegliere la soglia ogni volta procedendo similmente alla ricerca binaria. Quindi scelgo la soglia w come l'entrata centrale dell'array dei pesi. Studio l'esistenza di un cammino che collega u e v in un grafo con soli archi di peso minore o uguale a w . Se trovo un cammino, aggiorno w cercando nella metà sinistra (l'entrata centrale della metà sinistra), altrimenti aggiorno w con l'entrata centrale della metà di destra dell'array dei pesi.

In questo modo, ogni volta che aggiorno w devo fare una DFS e il costo dovrebbe essere quindi $O((n+m)\log(m))$, perché l'aggiornamento di w procede come in ricerca binaria, che ha costo $O(\log(m))$, e una DFS ha costo $O(n+m)$, dove n è il numero di nodi e m il numero di archi.

Con questo tipo di soluzione, anche se il costo asintotico non è troppo vantaggioso, se devo chiamare molte volte la funzione sono costretto a fare molte DFS. Potrebbe allora essere più conveniente costruire una volta sola il Minimum-cost Spanning Tree (MST) ed utilizzare quello, senza ricorrere al grafo. Infatti per MST vale la proprietà che il peso del cammino nel MST che collega due nodi u e v è il minimo che sto cercando.

Costruisco quindi il MST. A questo punto, essendomi ricondotto a un albero, scelgo una radice arbitraria, eseguo DFS e per ogni nodo salvo la profondità a cui si trova e il nodo padre. Poi costruisco una tabella degli antenati e una tabella che salvi il massimo peso incontrato sui cammini. Nella chiamata i nodi vengono portati alla stessa profondità e poi fatti risalire fino all'antenato comune più recente. Mantengo il massimo peso che incontro nella risalita e il valore finale è quello che sto cercando. (In questo commit non implemento questa versione migliorata in codice perché voglio convincermi che sia scritta bene e rendermi conto del costo)

3 Scelte di implementazione

3.1 Archi, nodi e grafo

- Per i nodi, voglio associare al numero identificativo di ogni AS un indice. La struct `Nodo` contiene solo il codice identificativo dell'AS, quella `Arco` l'intero che corrisponde all'AS collegato all'arco e il peso.
- Per la classe `Grafo`, inserisco la lista dei nodi del grafo in un vector, gli archi invece vengono salvati in una lista di adiacenza, in cui `adj[i]` contiene i vicini del nodo `i`.
- Dichiaro poi `idToIndex` che associa al numero identificativo di un AS un indice che uso internamente nel codice.
- E' necessario anche costruire una funzione che permette di attribuire la stessa chiave agli archi (u, v) e (v, u) , poiché il grafo è non orientato.

Nota: per la gestione delle tabelle hash utilizzo `unordered_map` per non preoccuparmi di rehashing. Potrei successivamente cambiare questa scelta e implementare le tabelle hash a mano.

Nota 2: nel commit del 16/09 è stata modificata la struct `Arco`, inserendo il peso all'interno, e quindi è stata rimossa la funzione `Frequenze`.

4 Analisi dei costi computazionali

4.1 ChiaveArco

Costa $O(1)$ perché fa operazioni elementari (confronti, swap, conversioni...).

4.2 AggiungiNodo

Utilizza `idToIndex`, che dichiaro come `unordered_map`, e ha come costo medio $O(1)$ (al caso pessimo potrebbe richiedere $O(n)$, con n il numero di nodi). Il `push_back` sui nodi e su `adj` ha come costo ammortizzato $O(1)$ e l'inserimento nella tabella hash ha ancora come costo medio $O(1)$ (al caso pessimo $O(n)$). Quindi la funzione ha un costo medio $O(1)$ (che al caso pessimo diventa $O(n)$).

4.3 AggiungiArco

Chiama due volte `AggiungiNodo`, che ha costo medio $O(1)$ e fa due accessi alla tabella hash, quindi sempre $O(1)$ come costo medio. Infine chiama `ChiaveArco`, che ha costo medio $O(1)$. Anche nella modifica del peso dell'arco c'è un costo medio di $O(1)$ perché sto usando una tabella hash per i pesi. Quindi il costo medio di `AggiungiArco` è $O(1)$.

4.4 Vicini

Questa è una funzione che mi restituisce il riferimento alla lista dei vicini di un nodo, che costa $O(1)$ mediamente perché sto accedendo a una tabella hash e a un elemento del vector `adj[indice]` tramite indice.